



UNIVERSAL ROBOTS

Methods of Knowledge Engineering

ArUco-guided UR7e robot system

Computer vision • coordinate frames •
robot control • LLM/RAG-based voice
selection



UR5e

Lightweight, versatile cobot



Reach
850 mm / 33.5 in



Payload
5 kg / 11 lbs



Footprint
Ø 149 mm

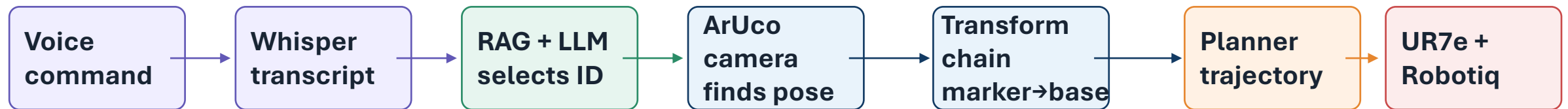


Weight
18,4 kg / 45.4 lbs

Dr eng. Michał Majewski

Whole pipeline: from request to physical action

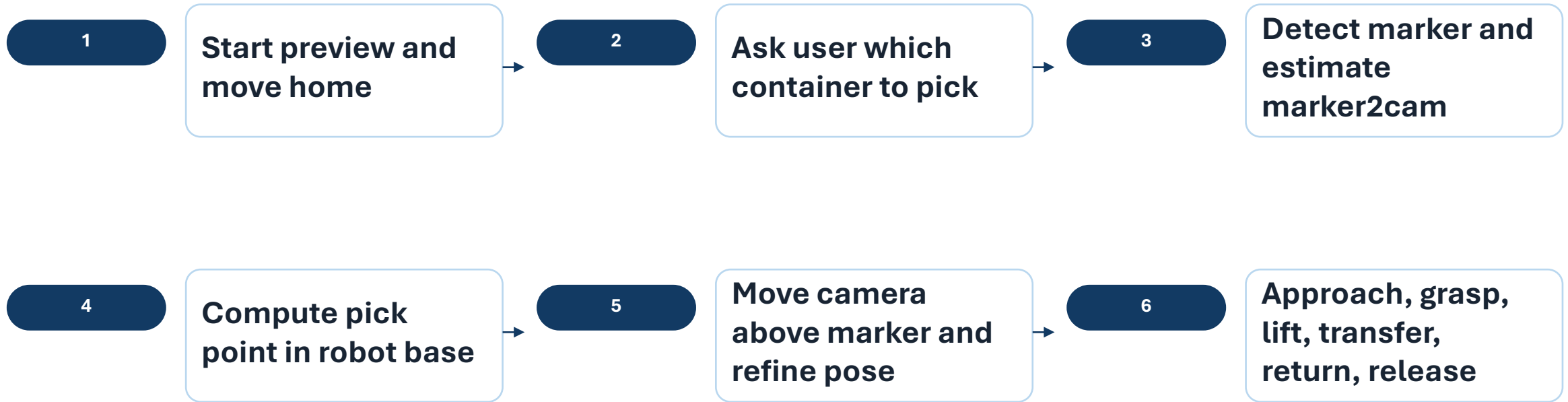
The system combines speech understanding, visual localization and robot execution.



- Knowledge part: natural-language intent → symbolic container ID.
- Vision part: marker pixels → 3D marker pose in camera coordinates.
- Geometry part: homogeneous transformations express the target in robot base coordinates.
- Execution part: safe joint moves, linear TCP moves and gripper commands.

Runtime sequence

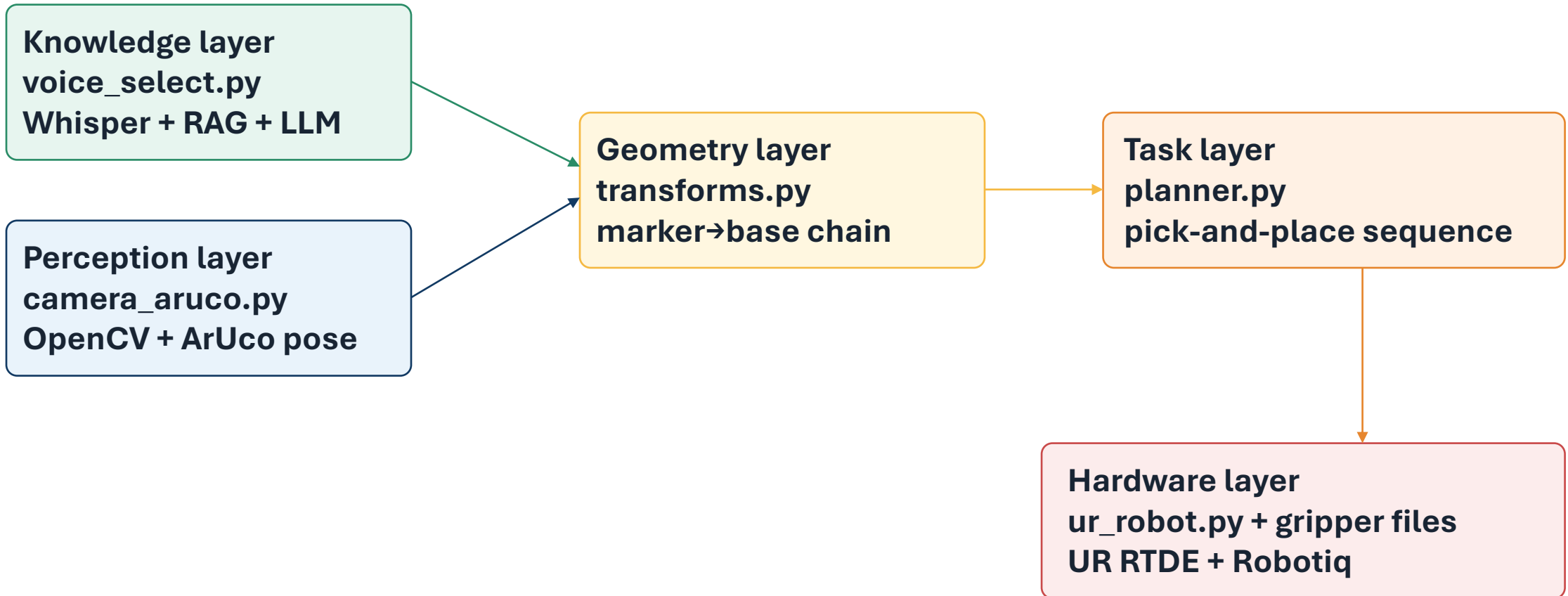
One iteration of the application loop.



**The loop is deliberately conservative:
after each task, the robot returns to the predefined HOME joint pose.**

System architecture

The modules are separated by responsibility: perception, knowledge, geometry and actuation.



main.py creates and connects all modules.

main.py — application entry point

Creates the key objects, connects hardware and starts the main loop.

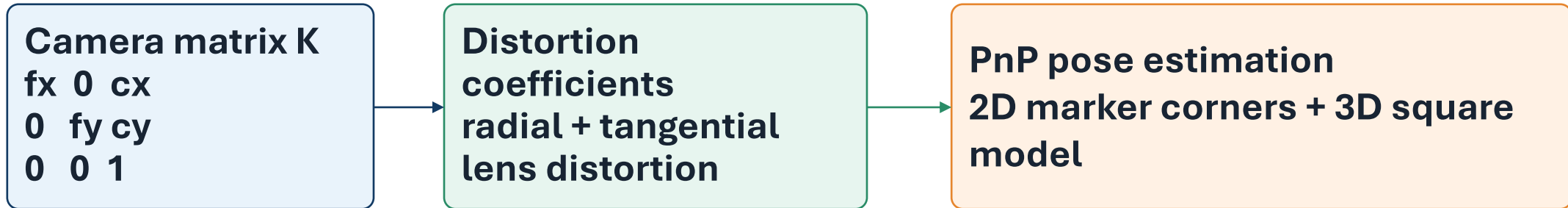
- Defines basic hardware parameters: robot IP, camera ID, marker length, gripper TCP, force and speed.
- Instantiates VoiceContainerSelector, ArucoCamera, Transformations, URRobot and Planner.
- Calls planner.run_forever(), then cleans up camera, robot and voice selector in finally.

```
voice_selector = VoiceContainerSelector()
cam = ArucoCamera(cam_id=CAM_ID,
marker_length=MARKER_LENGTH)
tf = Transformations("handeye_result.npz")
robot = URRobot(ROBOT_IP, tcp=GRIPPER_TCP,
...)
planner = Planner(cam, tf, robot,
voice_selector=voice_selector)
planner.run_forever()
```

**Idea: main.py is the composition root
it wires together perception, geometry, planning and execution.**

Camera calibration

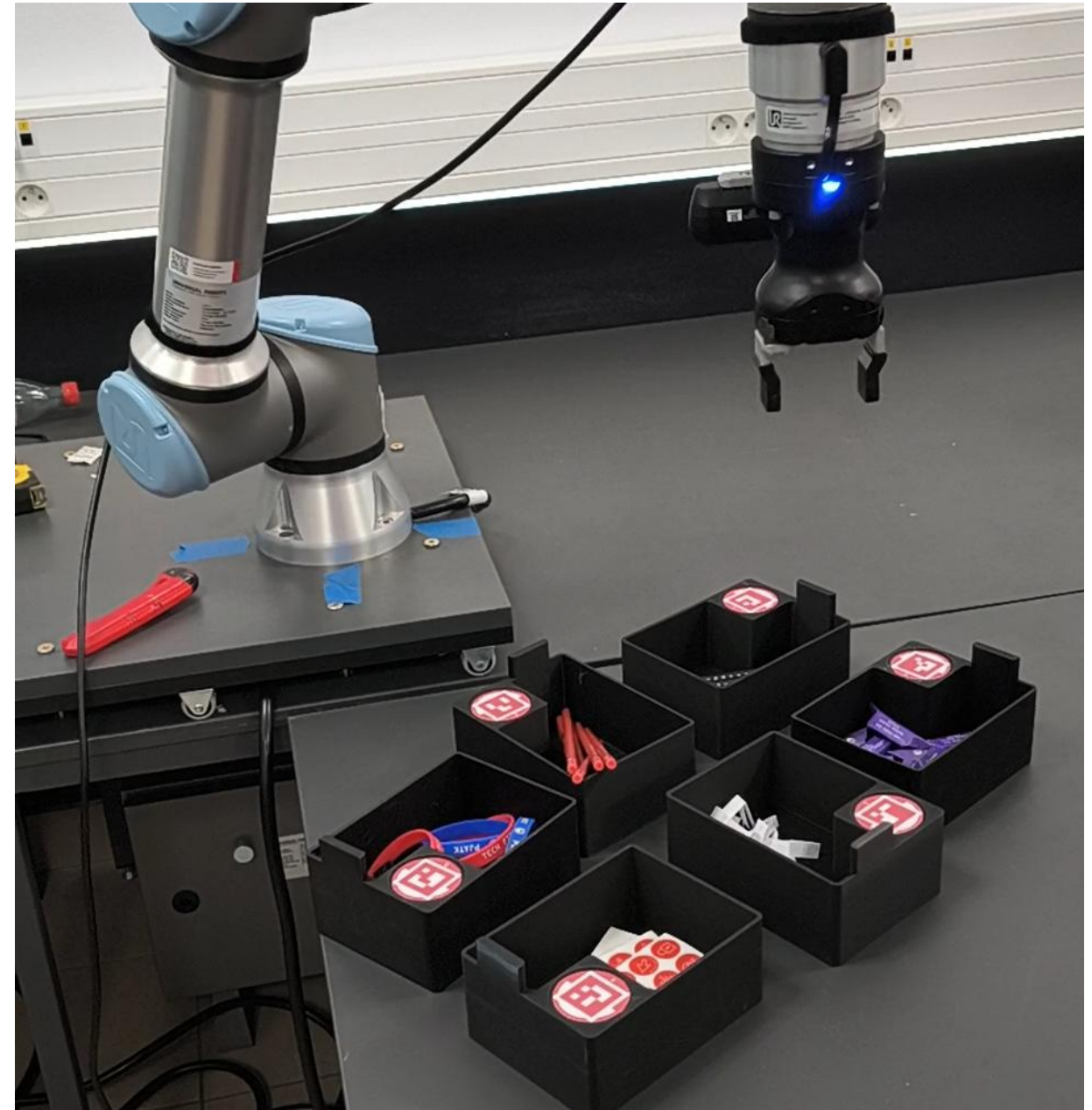
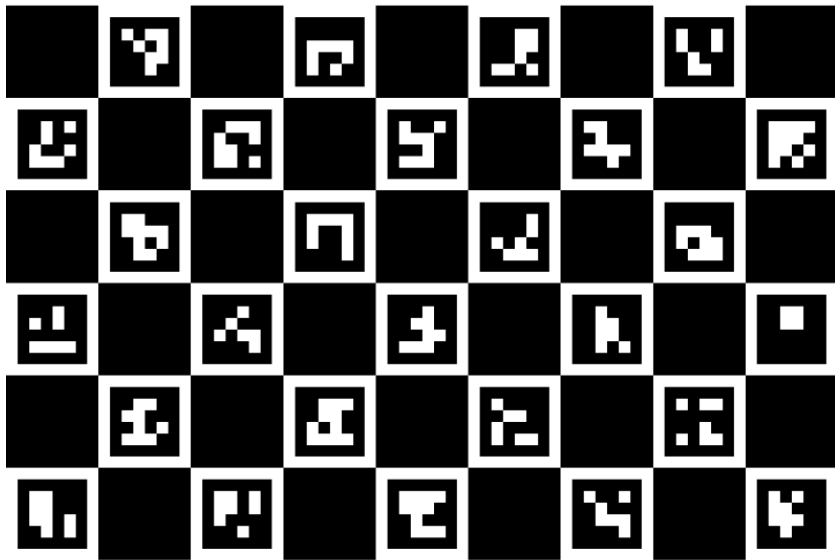
Calibration turns camera pixels into metrically meaningful 3D rays.



- camera_matrix.npy stores intrinsic parameters such as focal length and image center.
- dist_coeffs.npy stores lens distortion correction parameters.
- A calibrated camera can estimate T_marker2cam instead of only returning 2D marker pixels.

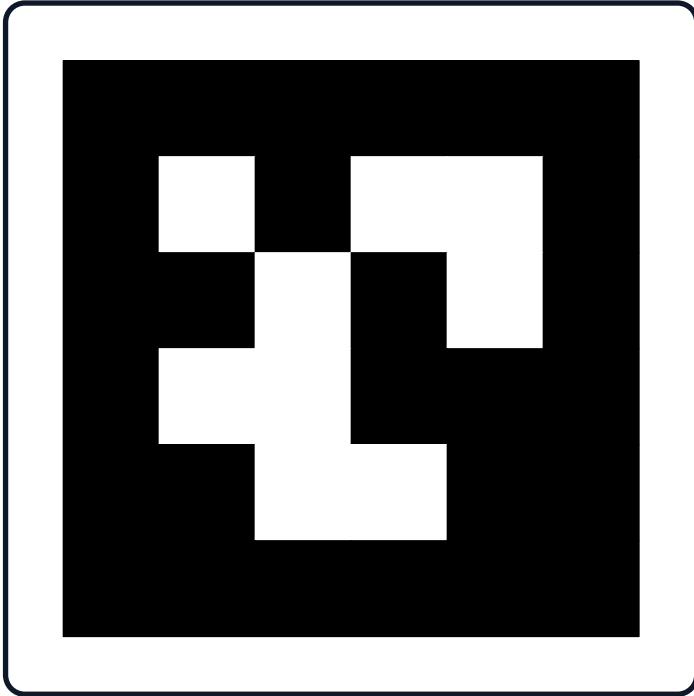
Camera calibration

Calibration turns camera pixels into metrically meaningful 3D rays.



What are ArUco codes?

ArUco markers are square fiducial markers used for fast camera-based localization.



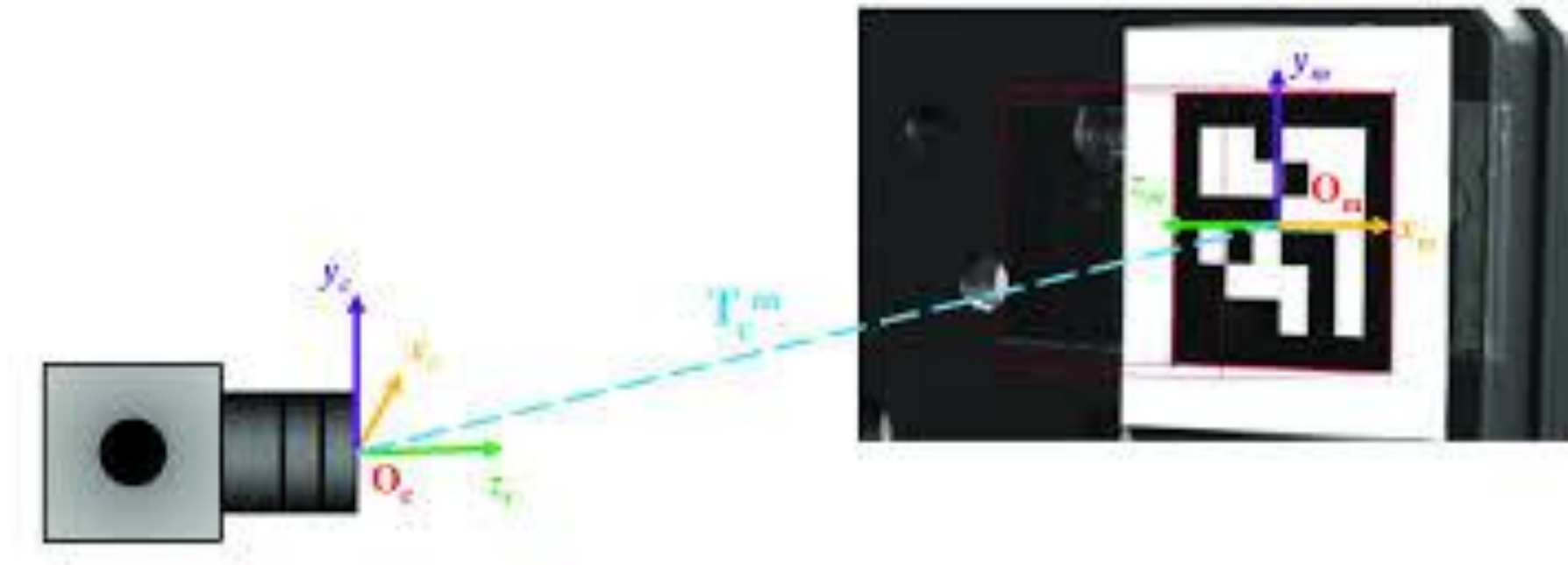
ID = 17

- The black border makes the square easy to detect.
- The internal binary pattern encodes an ID.
- Known marker size allows pose estimation: translation + rotation.
- In this project: DICT_4X4_50 and a physical marker length of 0.028 m.

Marker ID is not the object itself. It is a symbolic anchor attached to the physical container.

Coordinate frames: ArUco marker and camera

A pose is always a transformation between coordinate frames, not just a point.

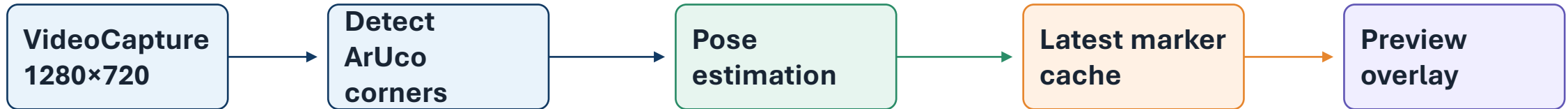


$T_{\text{marker2cam}}$ maps marker coordinates into camera coordinates.

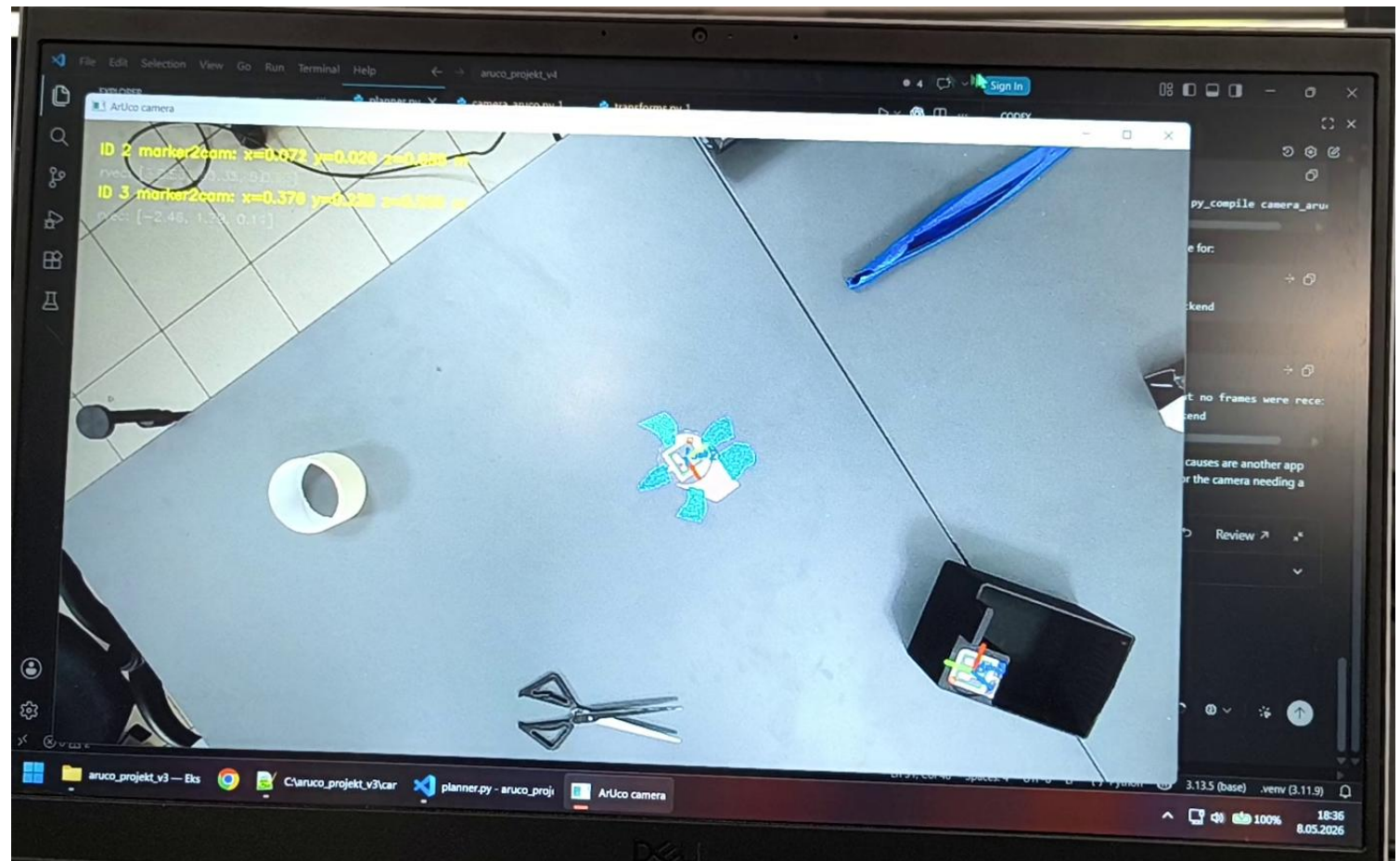
- Translation vector: where the marker center is, measured in meters in the camera frame.
- Rotation vector/matrix: how the marker axes are oriented relative to camera axes.

camera_aruco.py — computer vision module

Reads frames, detects markers and estimates their 3D pose relative to the camera.

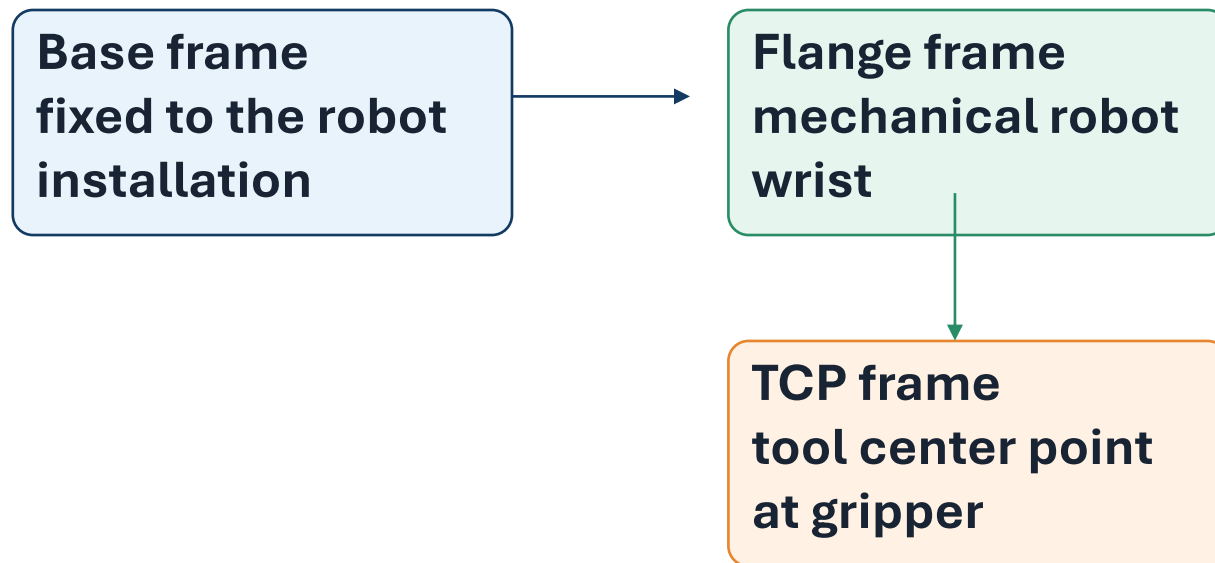


- Loads camera_matrix.npy and dist_coeffs.npy.
- Uses OpenCV ArUco dictionary DICT_4X4_50.
- Builds T_marker2cam from rvec/tvec.
- Runs preview in a background thread and stores latest marker data safely with a lock.



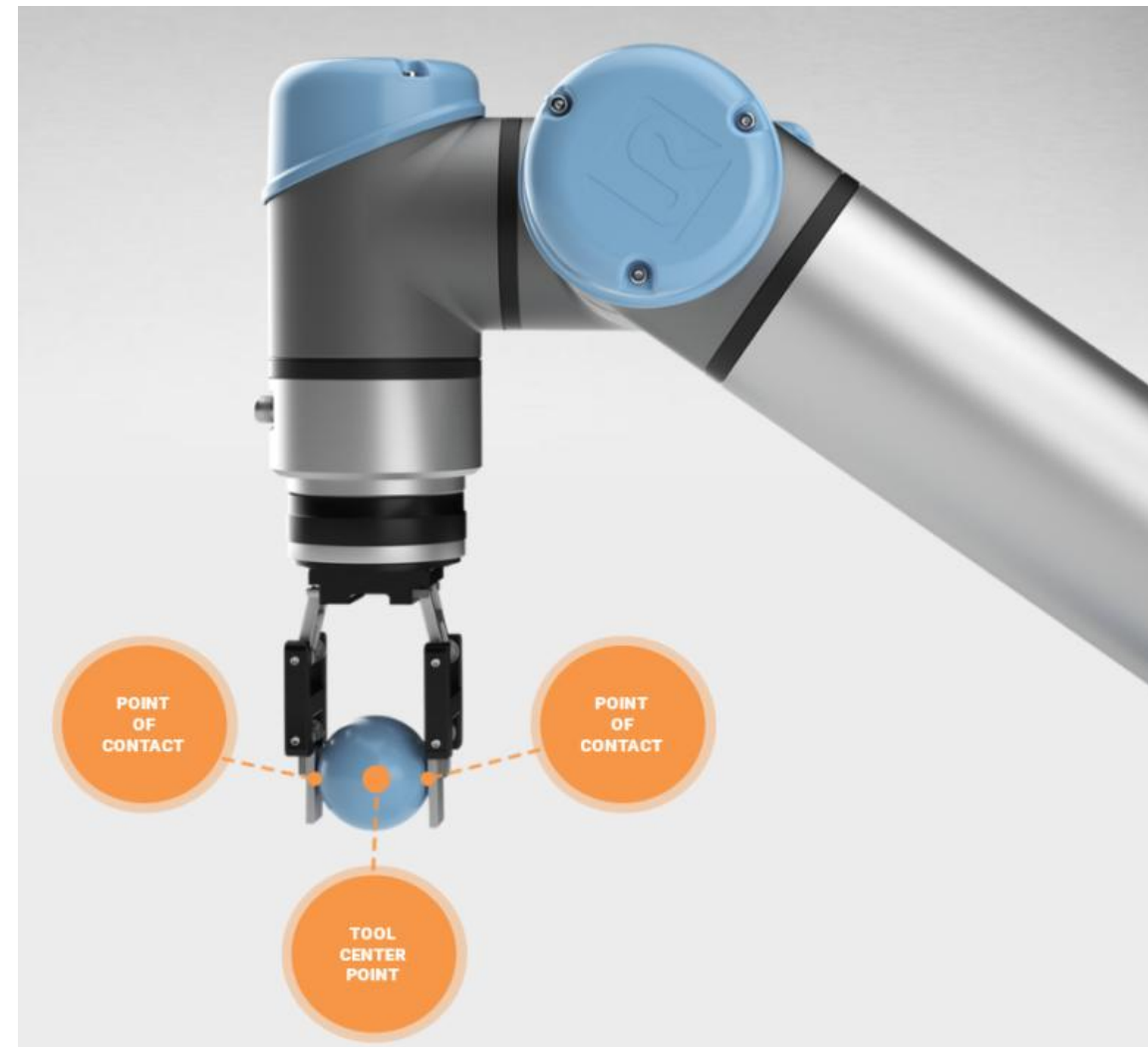
Robot frames: UR7e base, flange and TCP

Robot commands are expressed with respect to the robot base and the active TCP.



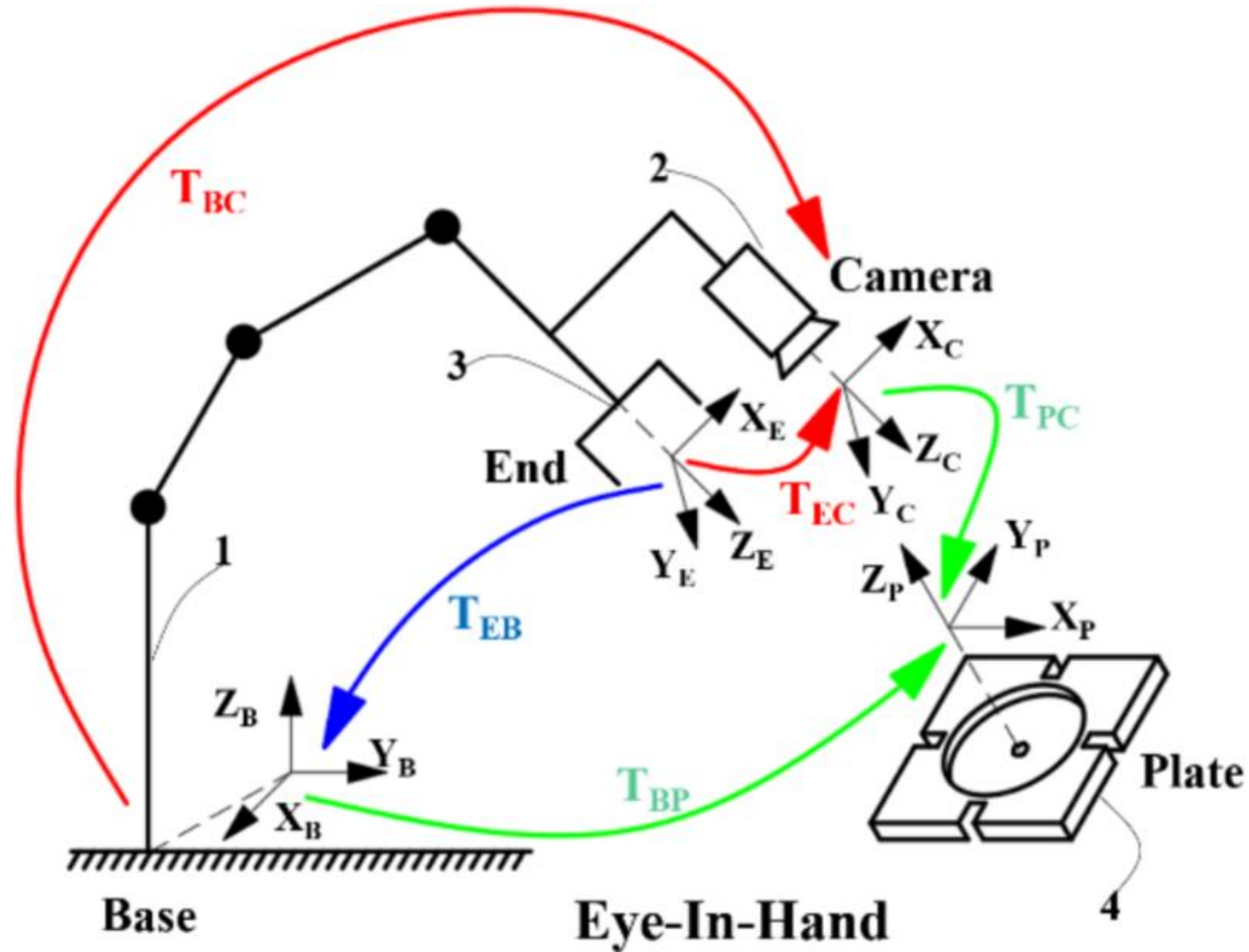
UR pose = [x, y, z, rx, ry, rz]

- x, y, z are translation in meters.
- rx, ry, rz are an axis-angle rotation vector.
- `getActualTCPPOSE()` returns TCP → base.
- Configured TCP describes tool offset from flange.



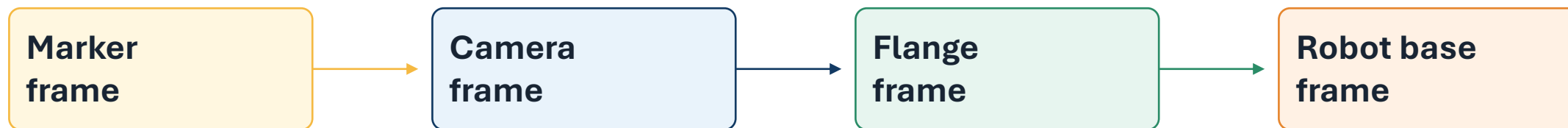
Transform chain: marker $\rightarrow$ camera $\rightarrow$ flange $\rightarrow$ base

This is the geometric core of the system.



transforms.py — coordinate transformation toolbox

Converts poses, matrices and points between the system coordinate frames.



$$\mathbf{T_marker2base} = \mathbf{T_flange2base} \cdot \mathbf{T_cam2flange} \cdot \mathbf{T_marker2cam}$$

- The marker is detected by the camera.
- Hand-eye calibration provides camera pose relative to the robot flange.
- The robot reports flange/TCP pose relative to base.
- The final pick point is expressed in base coordinates, so the robot can move to it.

Key idea: robot motion requires targets in base coordinates, but visual detection starts in camera coordinates.

planner.py — task logic and motion plan

Implements the high-level pick-and-place behavior.

Safe positions

HOME and TRANSFER joint poses are loaded from robot_positions.json.

User choice

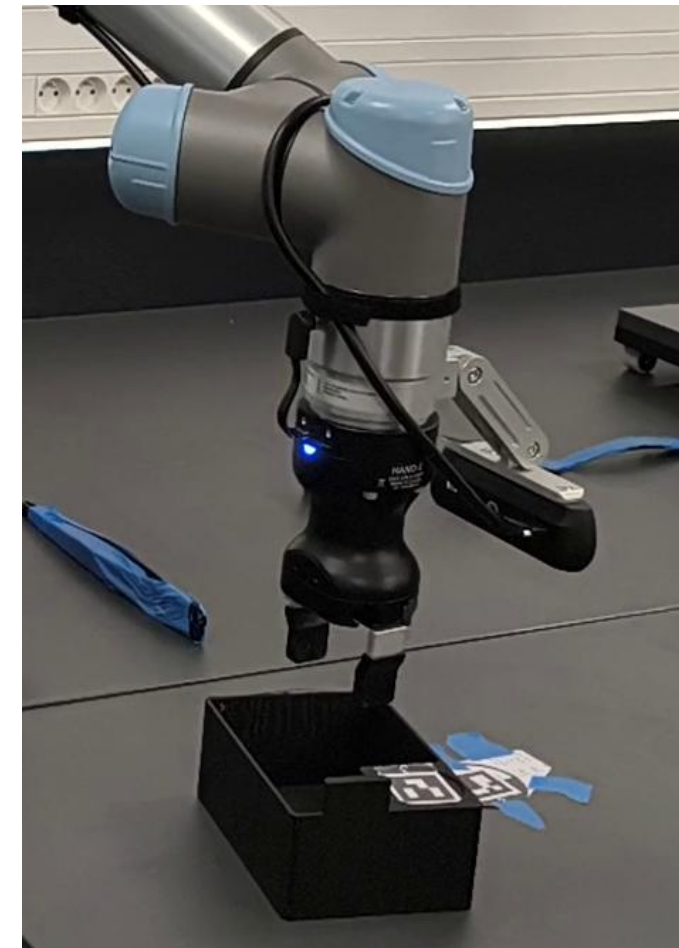
Container ID is selected through voice/RAG/LLM or manual input.

Pick point

Grip point is defined in marker coordinates and transformed into base coordinates.

Execution

Approach, open gripper, descend, close gripper, lift, transfer, return and release.



Planner = finite-state task controller, not a path optimizer.

ur_robot.py — UR7e interface

Encapsulates RTDE communication, TCP configuration and robot/gripper actions.

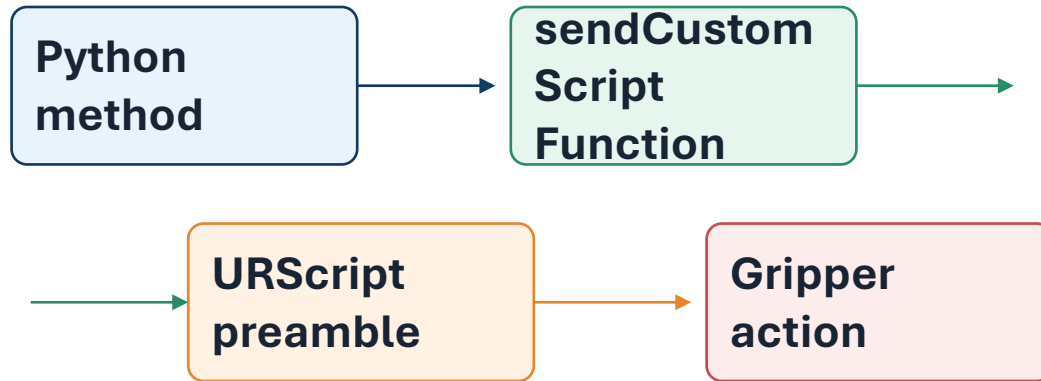
- Connects to RTDEControlInterface and RTDEReceiveInterface.
- Sets the active TCP and stores velocity/acceleration parameters.
- Provides moveJ for joint-space safe positions and moveL for Cartesian TCP motion.
- Computes flange2base from actual TCP pose and configured TCP offset.
- Wraps gripper activation, force/speed setup, open and close commands.

```
def get_flange2base(self):  
    T_tcp2base = self.get_tcp2base()  
    T_tcp2flange = self.get_tcp2flange()  
    return T_tcp2base @ inv(T_tcp2flange)
```

Interface role: hide low-level RTDE calls behind clear robot operations.

robotiq_gripper_control.py — Robotiq Hand-E wrapper

Converts Python calls into URScript commands sent through RTDE.

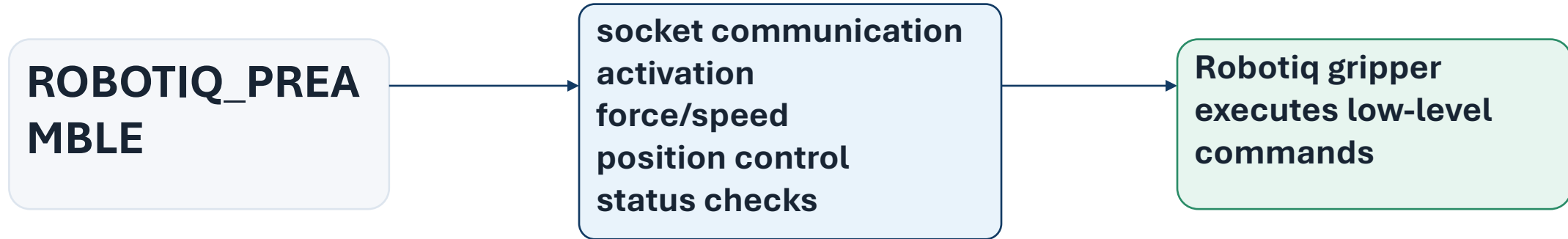


- Methods: `activate()`, `set_speed()`, `set_force()`, `move()`, `open()`, `close()`.
- Each method sends a custom script function such as `rq_open_and_wait()` or `rq_close_and_wait()`.
- The wrapper keeps the rest of the application independent from Robotiq URScript details.



robotiq_preamble.py — URScript support library

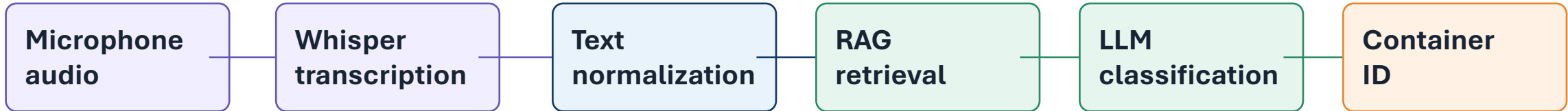
A large string containing the helper functions required by the Robotiq gripper.



- It defines URScript functions such as `rq_activate`, `rq_open_and_wait` and `rq_close_and_wait`.
- It handles socket access to the gripper server and low-level variable exchange.
- This file is infrastructure: do not usually modify it, but it is necessary for gripper control.

voice_select.py — speech, LLM and RAG selector

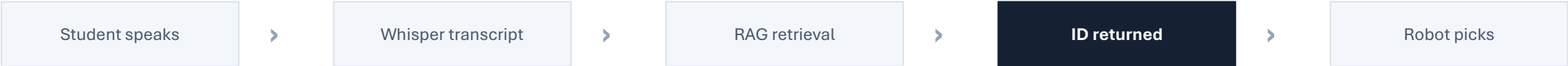
Maps a spoken user request to one of the known container IDs.



- Stores documents for containers: candies, pens and quote cards.
- Scores text with lexical retrieval, then optionally calls an Ollama LLM.
- Contains a fallback path when Ollama is unavailable.
- Returns a strict ID: 0, 1, 2 or None.

voice_select.py — speech, LLM and RAG selector

Open Days demo: the RAG selector maps a spoken request to one numeric container ID.



ID	Container	Request intent	Example English commands
0	PJATK candies	sweet snack, treat, candy	“I want something sweet.” • “Can I get some candy?”
1	Pens	writing, notes, signature, phone number	“I need a pen.” • “I want to write down a phone number.”
2	Quote cards	motivation, inspiration, quote, thought	“Give me a motivational quote.” • “I need some inspiration.”
3	Drawing pencils	drawing, sketching, pencil, random item	“I want to draw something.” • “Give me a random product.”
4	Stickers	stickers, labels, decorating, marking	“I want stickers.” • “Give me something to decorate with.”
5	Clickers / keychains / wristbands	fidget toy, stress relief, calming item	“I want a fidget toy.” • “Give me something calming.”

Technical note: the system should output only one value: 0, 1, 2, 3, 4 or 5. “Random product / random item” is intentionally mapped to ID 3.

Summary

The project is a compact example of knowledge engineering in a physical AI system.

Perception

ArUco markers provide a bridge from pixels to geometry.

Knowledge

RAG/LLM maps natural language to a symbolic container ID.

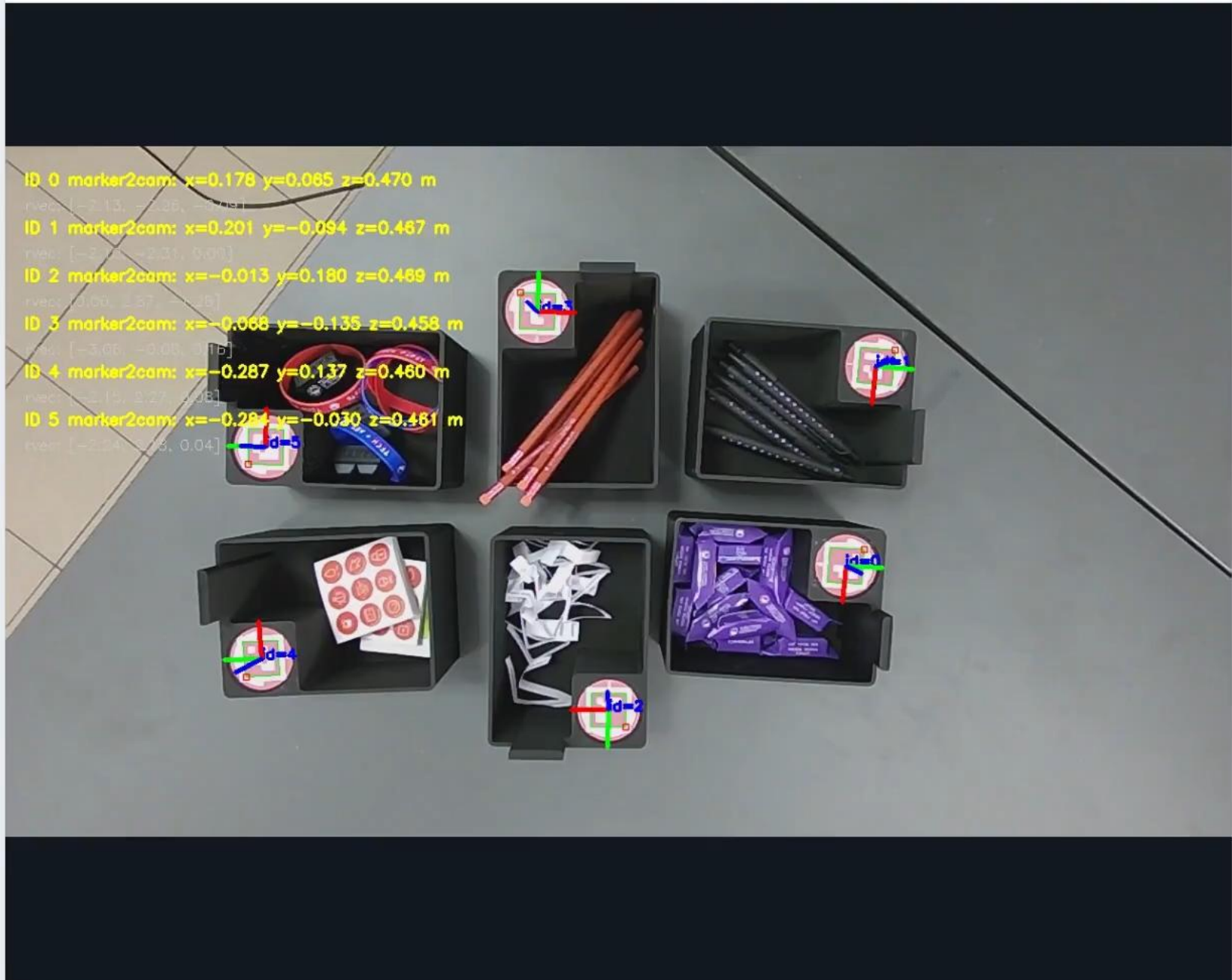
Geometry

Coordinate transforms convert camera observations into robot-base targets.

Action

UR7e and Robotiq execute a safe pick-and-place routine.

From sentence → container ID → marker pose → base pose → TCP motion.



AI Agent

● Ready

Ready for a voice command

Start Listening

Listening

Transcribing

Choosing

Result

AI QUERY

Chcę zapisać numer telefonu.

CANDIDATES

ID=1 Pens | ID=3 Drawing pencils



POLISH-JAPANESE
ACADEMY OF INFORMATION
TECHNOLOGY